Ex.No.5	Experiments on Packet capture tool: Wireshark
Date	

AIM:

To understand the features of wireshark as a packet capture tool and understand **encapsulation of information at various layers of a Protocol stack**.

WIRESHARK

Wireshark is a free, open-source network packet analyzer used for network troubleshooting, analysis, and security auditing. It lets you capture and interactively browse live data traffic passing through a network interface:

Workdone by Wireshark:

- Capture network traffic
- Decode packet protocols using dissectors
- Define filters – capture and display
- Watch smart statistics
- Analyze problems
- Interactively browse that traffic

Wireshark used for:

- Network administrators: troubleshoot network problems
- Network security engineers: examine security problems
- Developers: debug protocol implementations
- People: learn **network protocol internals**

Getting Wireshark

Wireshark can be downloaded for Windows or macOS from its official website. For Linux or another UNIX-like system, Wireshark will be found in its package repositories. For Ubuntu, Wireshark will be found in the Ubuntu Software Center.

Download Wireshark from the following link

https://www.wireshark.org/download.html

Procedure to Capture Packets Using Wireshark

Step 1: Download and install **Wireshark** on your computer.

Step 2: Open the **Wireshark** application.

Step 3: From the home screen, select the **network interface** (e.g., Wi-Fi or Ethernet) through which your computer is connected to the network.

Step 4: Click the **Start Capturing Packets** button (blue shark fin icon) or double-click the selected interface.

Step 5: Generate network traffic by opening a website, sending a ping, or browsing the Internet.

Step 6: Observe the captured packets appearing in real time in the Packet List pane.

Step 7: To view only specific packets, enter a **display filter** (e.g., http, tcp, udp, icmp, dns) in the filter bar and press **Enter**.

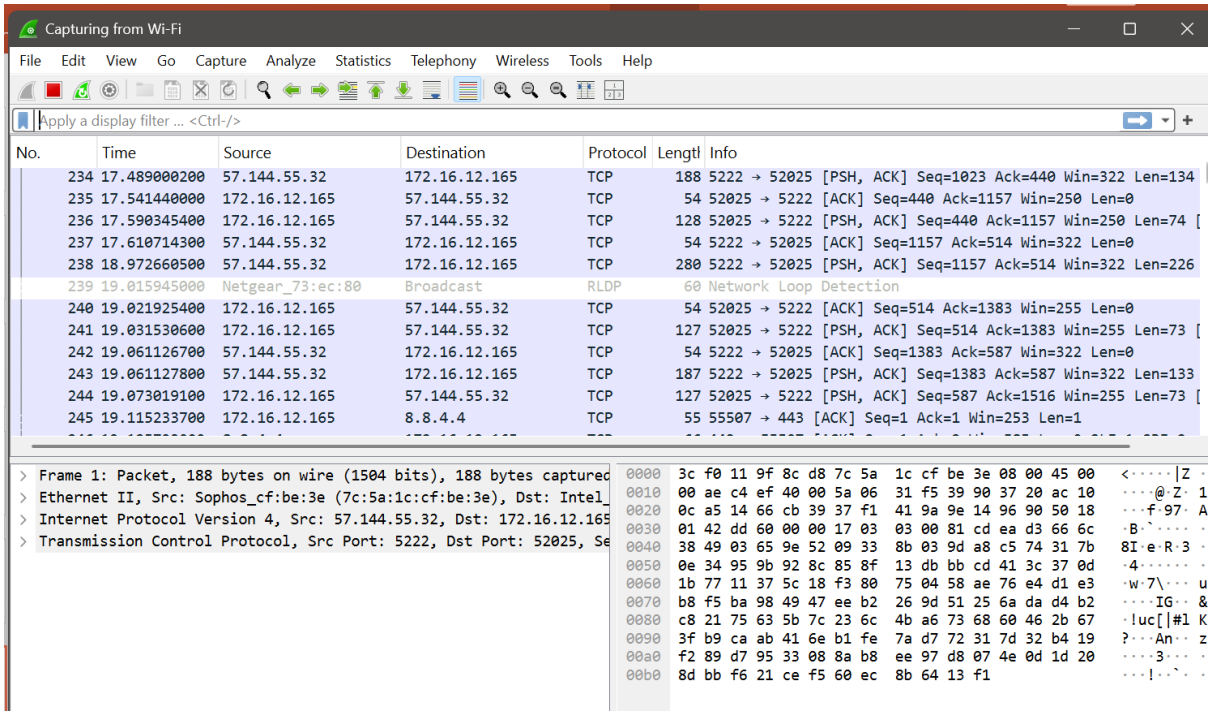
Step 8: Click on any packet to view its detailed information in the **Packet Details** pane.

Step 9: Expand the protocol layers (Ethernet, IP, TCP/UDP, etc.) to examine packet contents.

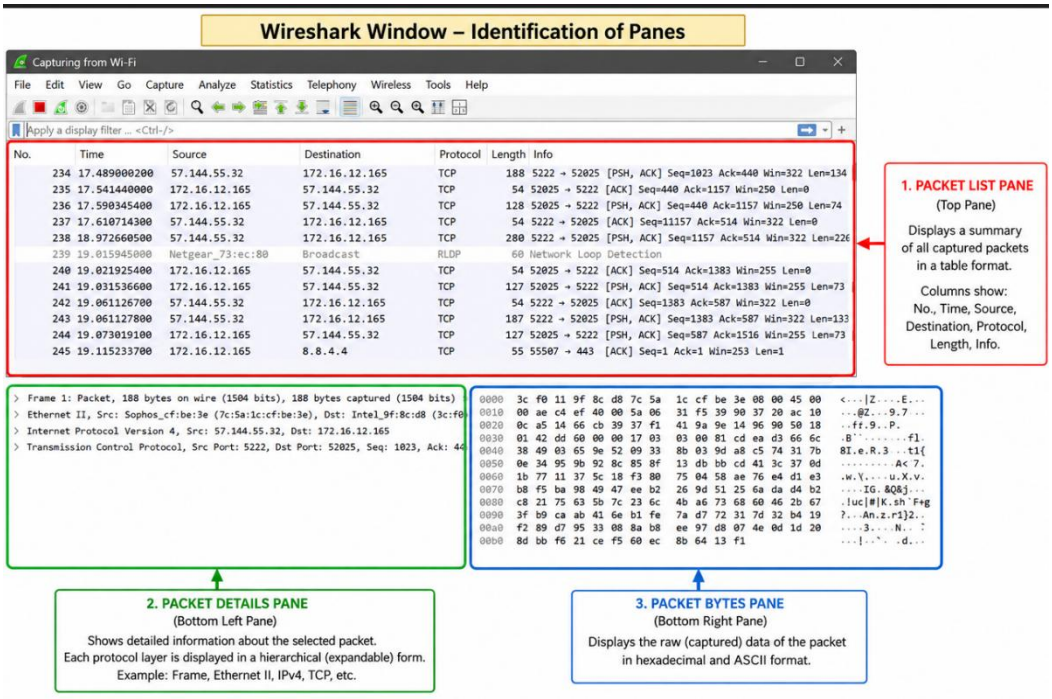
Step 10: Click the **Stop** button (red square) to stop packet capture.

Step 11: To save the captured packets, go to **File → Save As**, choose a location, and save the file in **.pcapng** (or **.pcap**) format.

Sample Capturing Packets from Wireshark: Sample output



Different Panes in Wireshark:



1. Packet List Pane (Top Pane)

- Displays a summary of all captured packets.
- Each row represents one captured packet.
- Shows details such as **Packet Number, Time, Source, Destination, Protocol, Length,** and **Info**.

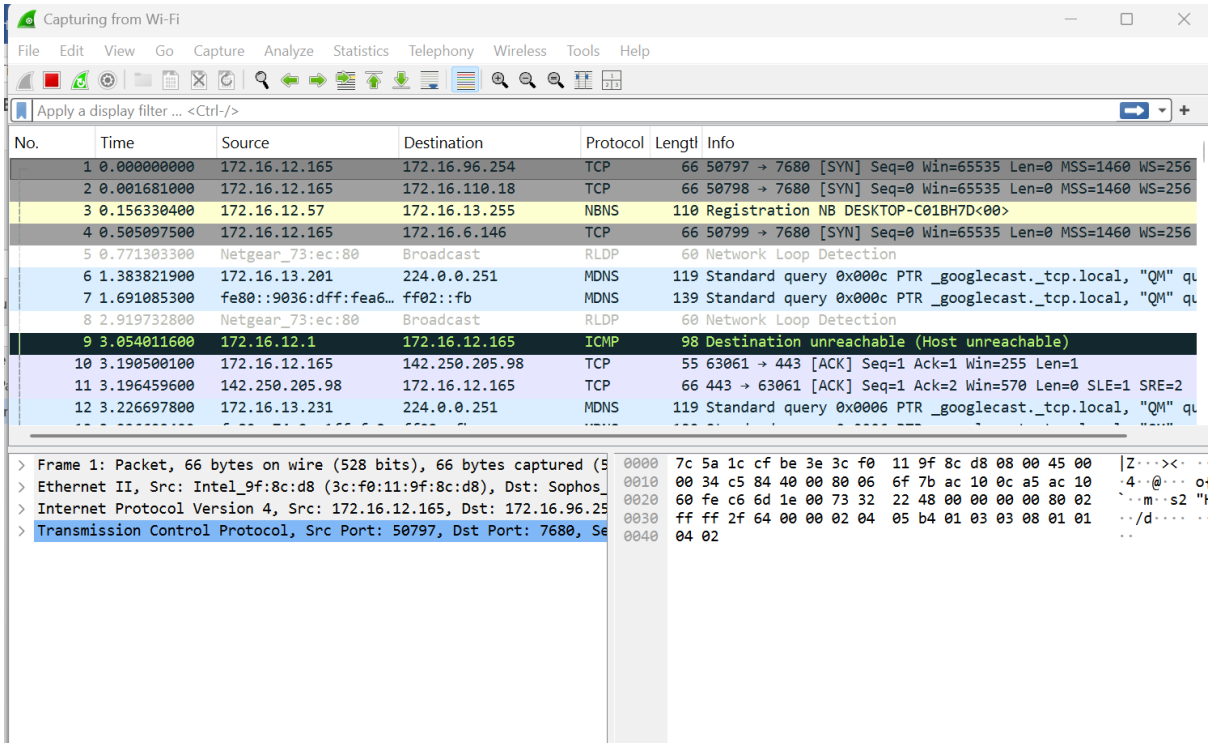
2. Packet Details Pane (Middle Pane)

- Displays detailed information about the selected packet.
- Information is organized into protocol layers such as **Frame, Ethernet, IP, TCP/UDP**, etc.
- Each protocol layer can be expanded to view more details.

3. Packet Bytes Pane (Bottom Pane)

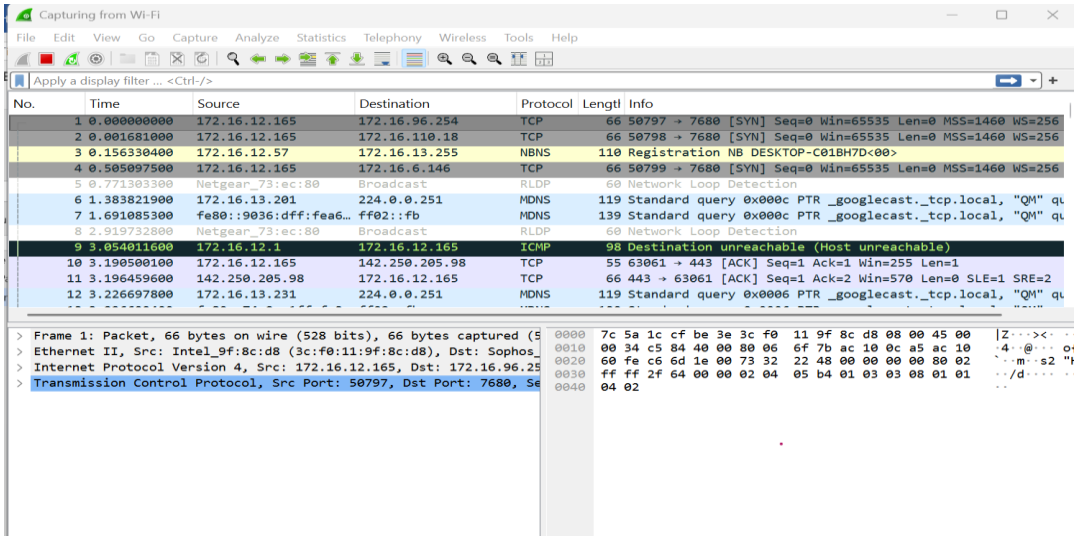
- Displays the raw data of the selected packet.
- Data is shown in **hexadecimal** format on the left.
- The corresponding **ASCII** representation is shown on the right.

Note: These three panes work together to help users analyze captured network packets efficiently in Wireshark.



Color Coding:

In Wireshark, **packet colors** help you quickly identify different types of network traffic. Based on your screenshot, the colors indicate the following:

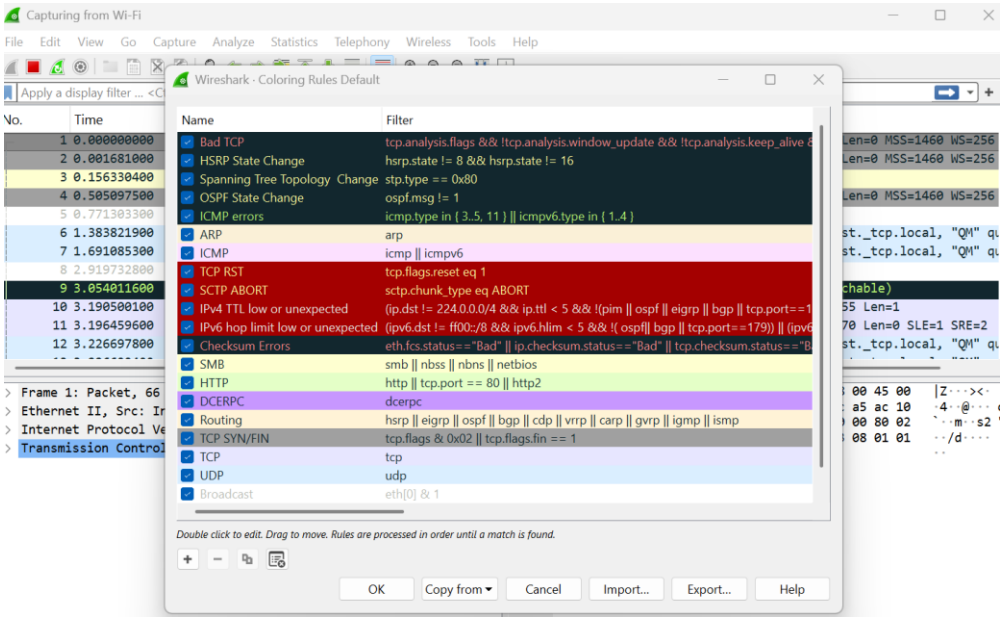


Color	Protocol/Traffic	Description
Light Gray	TCP	Indicates normal TCP packets such as SYN and ACK packets.
Light Yellow	NBNS (NetBIOS Name Service)	Used for NetBIOS name registration and name resolution on a local network.

Color	Protocol/Traffic	Description
White	RLDP (Network Loop Detection)	Represents Network Loop Detection Protocol packets used by network devices.
Light Blue	MDNS (Multicast DNS)	Used for local network service discovery (e.g., Chromecast, printers, Apple devices).
Dark Green / Black with Green Text	ICMP	Indicates Internet Control Message Protocol packets such as **Destination Unreachable**, **Ping Request**, or **Ping Reply**.
Light Purple / Lavender	TCP	Represents another category of TCP packets (e.g., ACK or established connection traffic) based on Wireshark's default coloring rules.

To view exactly what the color codes mean, click **View > Coloring Rules.** You can also customize and modify the coloring rules from here, if you like.

Sample Output:



Sample Captures:

- If there is no interesting network traffic on your own network, you can use **sample capture files**.
- Download a sample capture file and open it by selecting **File → Open** in Wireshark.

- Browse to the downloaded file and click **Open** to analyze the captured packets.
- You can also save your own captured packets for future analysis by selecting **File → Save**.
- Wireshark saves capture files in **.pcap** or **.pcapng** format.

Simple Procedure:

1. Download a sample capture file.
2. Open Wireshark.
3. Click **File → Open**.
4. Select the sample capture file.
5. Analyze the captured packets.
6. To save your own capture, click **File → Save**.

You can also save your own captures in Wireshark and open them later. Click **File > Save** to save your captured packets

Filtering Packets:

- Close unnecessary applications before capturing packets.
- This reduces unwanted network traffic.
- Capture only the traffic related to the program you want to analyze.
- Without filters, thousands of packets may be captured.
- Filters help narrow down the packet list.
- This makes packet analysis faster and easier.
- Wireshark provides **Capture Filters** and **Display Filters** to simplify traffic analysis.

Sample filter applied for TCP:

Filtered Packet List

No.	Time	Source	Destination	Protocol	Length	Info
1	0.001	192.168.1.10	142.250.183.78	TCP	74	50234 → 443 [SYN]
2	0.005	142.250.183.78	192.168.1.10	TCP	74	443 → 50234 [SYN, ACK]
3	0.006	192.168.1.10	142.250.183.78	TCP	66	50234 → 443 [ACK]

No.	Time	Source	Destination	Protocol	Length	Info
4	0.020	192.168.1.10	142.250.183.78	TCP	1514	HTTP GET Request
5	0.055	142.250.183.78	192.168.1.10	TCP	1514	HTTP 200 OK

Display Filters in Wireshark:

- Click **Analyze → Display Filters** to open the list of available display filters.
- Choose a **predefined filter** from the default list.
- You can **create your own custom filters**.
- Save custom filters for **future use**.
- Display filters make it easier to find specific packets.
- For advanced filter syntax, refer to the **official Wireshark Display Filter documentation**.

Example: Create and Save a Custom Filter

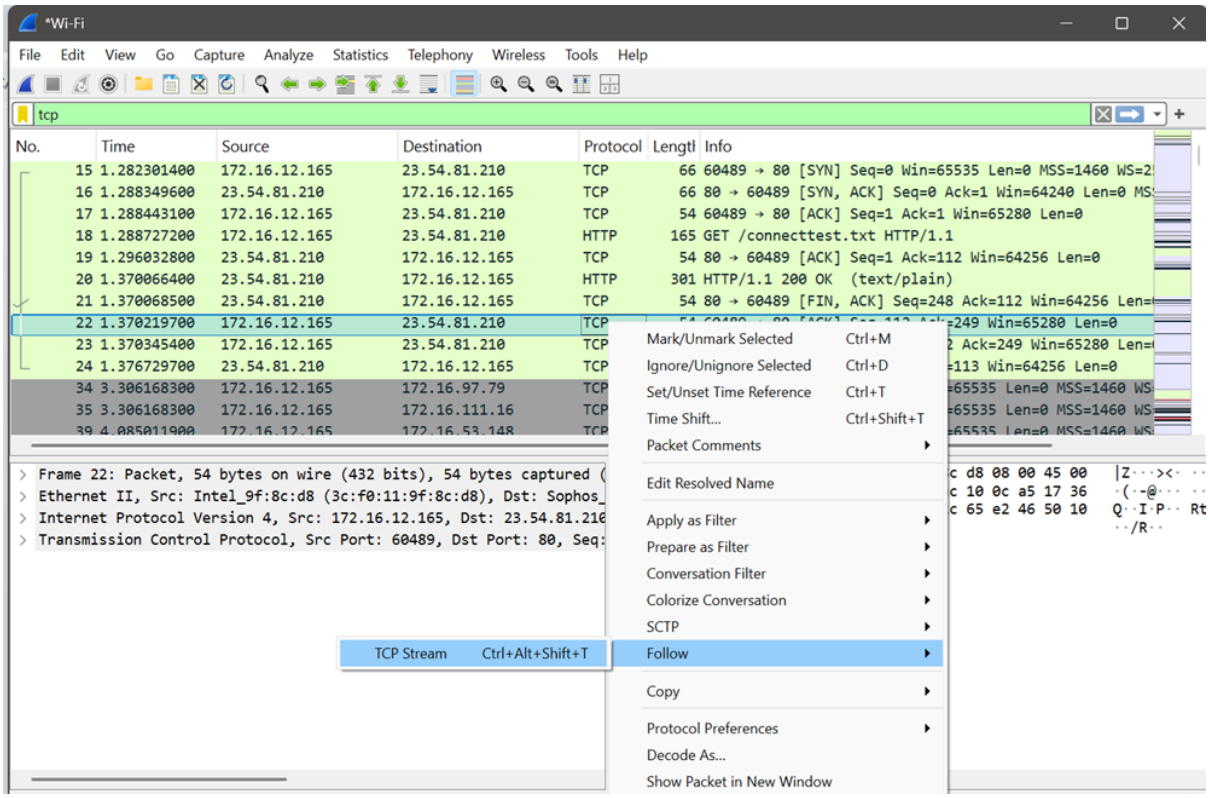
Suppose you frequently analyze **DNS packets**.

1. Click **Analyze → Display Filters**.
2. Click the **+ (Add)** button.
3. Enter:
    - **Filter Name:** DNS Packets
    - **Filter Expression:** dns
4. Click **OK**.

Follow TCP Stream in Wireshark

1. Right-click on a **TCP packet**.
2. Select **Follow → TCP Stream**.
3. Wireshark displays the **complete conversation** between the client and the server.
4. This helps you understand the **data exchanged** during the communication.
5. You can also choose **Follow → UDP Stream** or other available protocol streams, if supported.

6. This feature is useful for analyzing web requests, application data, and troubleshooting network communication.



Sample Output:

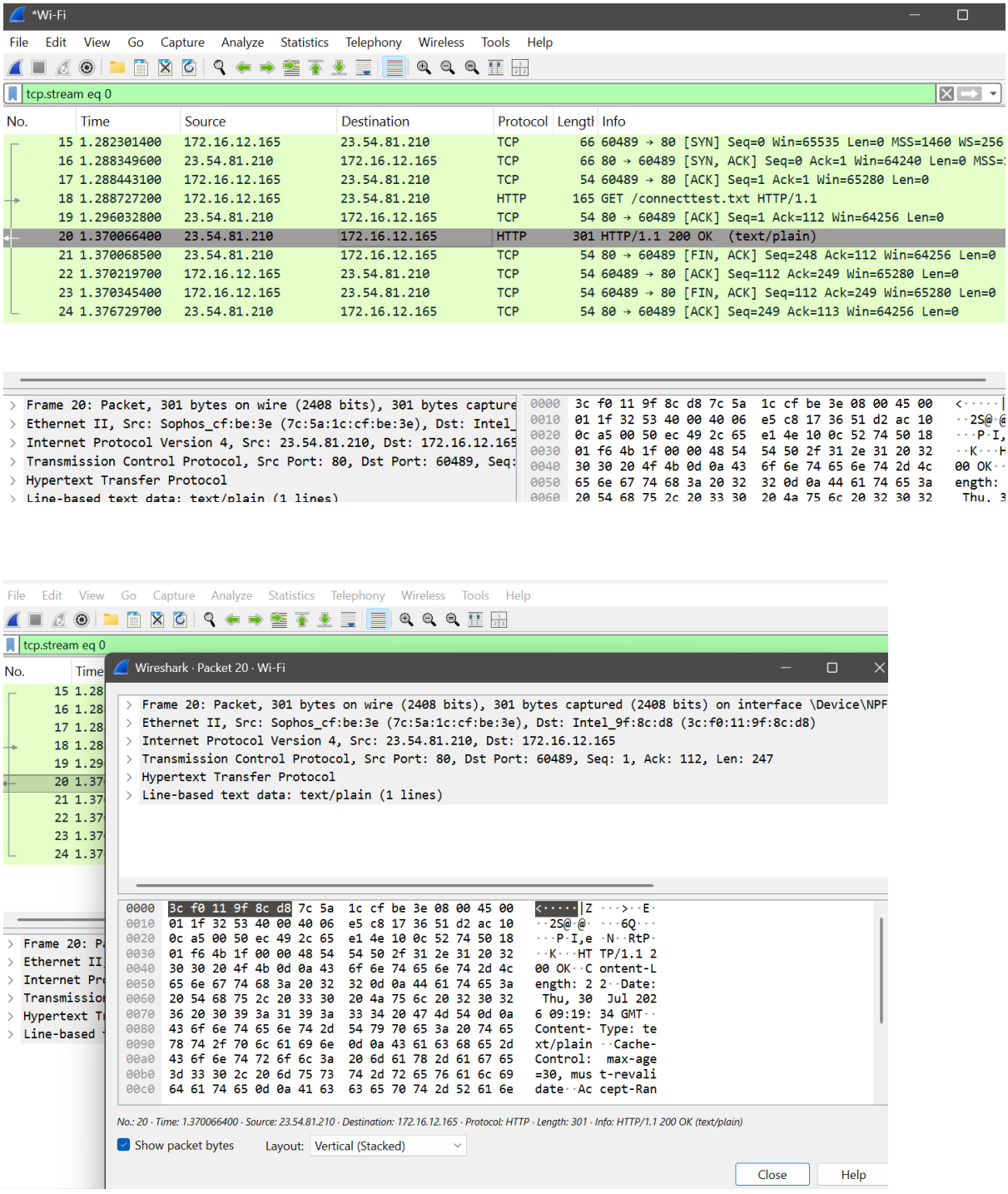
No.	Source	Destination	Protocol	Info
35	192.168.1.10	93.184.216.34	TCP	50321 → 80 [SYN]
36	93.184.216.34	192.168.1.10	TCP	80 → 50321 [SYN, ACK]
37	192.168.1.10	93.184.216.34	TCP	50321 → 80 [ACK]
38	192.168.1.10	93.184.216.34	HTTP	GET /index.html
39	93.184.216.34	192.168.1.10	HTTP	HTTP/1.1 200 OK

Inspecting Packets:

- Click a **packet** in the Packet List pane.
- The packet details are displayed in the **Packet Details** pane.
- Expand each protocol layer (Ethernet, IP, TCP/UDP, etc.) to view more information.

- View the raw packet data in the **Packet Bytes** pane.
- Inspecting packets helps analyze **source and destination addresses, protocol information, ports, and packet contents**.

Sample Output:



Flow Graph in Wireshark

Definition:
A **Flow Graph** visually represents the sequence of communication between different devices (hosts) in a network. It makes it easier to understand how packets are exchanged.

How to Open Flow Graph

1. Open a packet capture (.pcap) file.
2. Click **Statistics → Flow Graph**.
3. A Flow Graph window appears showing the communication between hosts.

What the Graph Shows

- **Vertical lines** represent different network devices (hosts).
- **Horizontal arrows** represent packets sent between hosts.
- **Time** is shown from **top to bottom**.
- Each arrow is labeled with the **protocol** and **packet information**.

Example:

Yes. Below is a **sample Flow Graph output** similar to what you would see in Wireshark after selecting **Statistics → Flow Graph**.

Sample Output: Flow Graph

```
Time      Client (192.168.1.10)       Server (93.184.216.34)

0.000     |--------------------------->|  DNS Query
0.015     |<---------------------------|  DNS Response

0.030     |--------------------------->|  TCP SYN
0.035     |<---------------------------|  TCP SYN-ACK
0.040     |--------------------------->|  TCP ACK

0.050     |--------------------------->|  HTTP GET /index.html
0.120     |<---------------------------|  HTTP/1.1 200 OK
```

0.125 |<----------| HTML Page Data

0.150 |---------->| TCP FIN

0.155 |<----------| TCP ACK

0.160 |<----------| TCP FIN

0.165 |---------->| TCP ACK

Wireshark Capture Options:

- Go to capture option
- Select stop capture automatically after 100 packets.
- Then click Start capture.

Step 1: Select the Active Interface

In your screenshot, choose the interface that is currently connected to the network.

- **If you are using Wi-Fi**, click **Wi-Fi**.
- **If you are using a LAN cable**, click **Ethernet**.

Tip: The active interface usually shows a moving graph or increasing traffic. In your screenshot, **Wi-Fi** is likely the correct choice if you are connected wirelessly.

Step 2: Go to the Options Tab

Click the **Options** tab at the top.

You will see settings similar to this:

☑ Stop capture automatically after

Step 3: Set the Packet Limit

Check the option:

☑ Stop capture automatically after

Select:

100 Packets

Creating a Filter to display only DHCP packets and inspect the packets.

Procedure

1. Open **Wireshark**.
2. Select the **Ethernet (Local Area Connection)** or **Wi-Fi** interface.
3. Click **Capture → Options**.
4. Select **Stop capture automatically after 100 packets**.
5. Click **Start** to begin packet capture.
6. Generate DHCP traffic by disconnecting and reconnecting the network, or by running:
7. ipconfig /release

ipconfig /renew

8. After the capture stops, type the following in the **Display Filter** bar:

dhcp

(If dhcp does not work, use bootp.)

9. Press **Enter** or click **Apply** to display only DHCP packets.

10. Click any DHCP packet to inspect its details.

11. Click **File → Save As**, enter a filename, and save the captured packets as a **.pcapng** file.

Create a Filter to display only IP/ICMP packets and inspect the packets. Procedure

Step 1: Check if ping is working

1. Press **Windows + R**.
2. Type **cmd** and press **Enter**.
3. Type:

ping 8.8.8.8

4. Press **Enter**.

What do you see?

- If you see:

Reply from 8.8.8.8...

then the ping is working.

- If you see:

Request timed out

or

Ping request could not find host...

tell me exactly what message appears.

Step 2: Check whether Wireshark is capturing

When you click **Start Capture**, do you see the packet count increasing continuously?

- **Yes** → Wireshark is capturing.
- **No** → The wrong network interface is selected.

Step 3: Verify the filter

After capturing packets, clear any existing filter and type:

icmp

If the filter bar turns **green**, the filter is valid.

If it stays green but shows **0 packets**, then no ICMP packets were captured.

Step 4: Easier alternative

Instead of using ping, try this:

1. Start Wireshark.
2. Open **Google Chrome**.
3. Visit **www.google.com**.
4. Stop the capture.
5. In the Display Filter, type:

dns

or

tcp

6. Press **Enter**.

These filters almost always show packets because web browsing generates DNS and TCP traffic.

Sample Output (ICMP Filter)

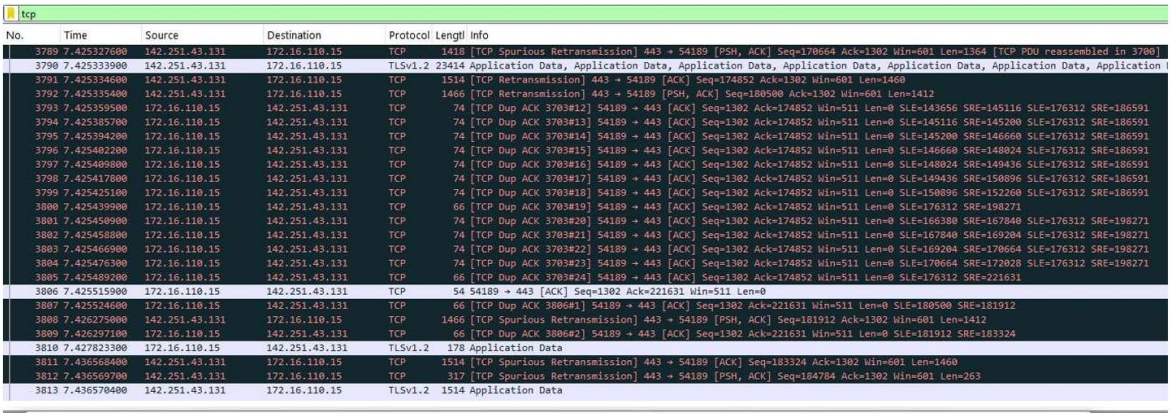
Display Filter:

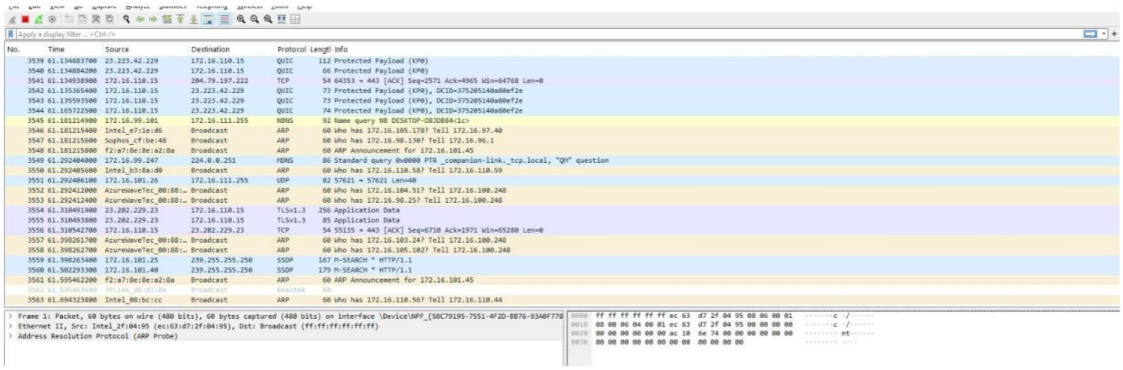
icmp

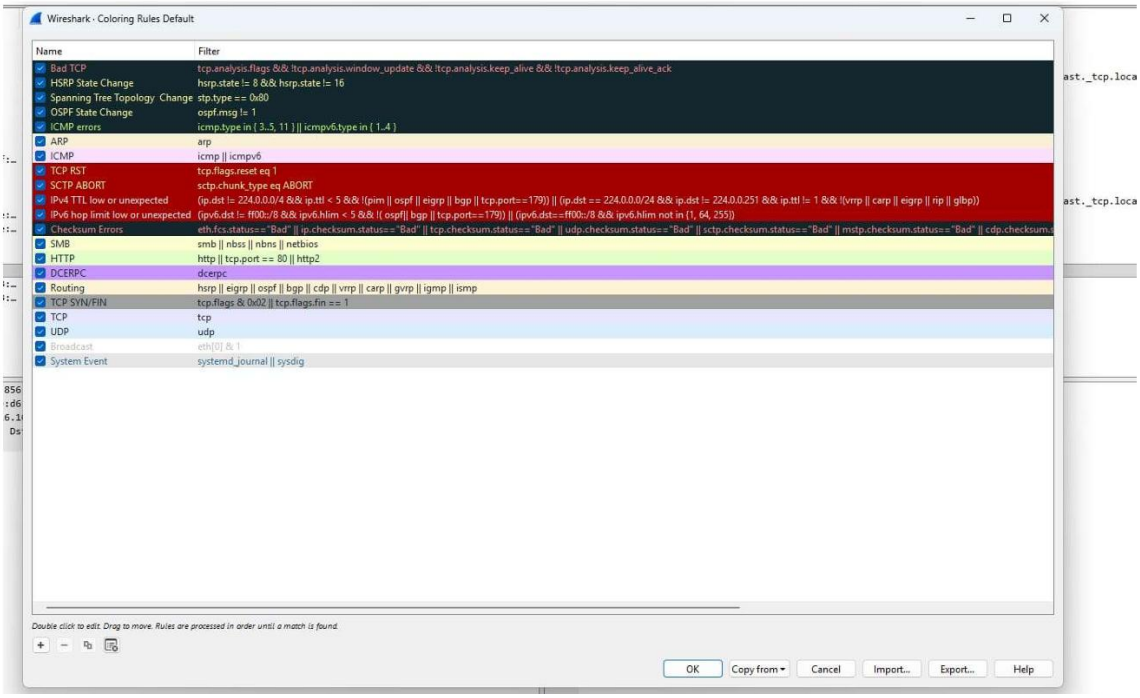
Packet List

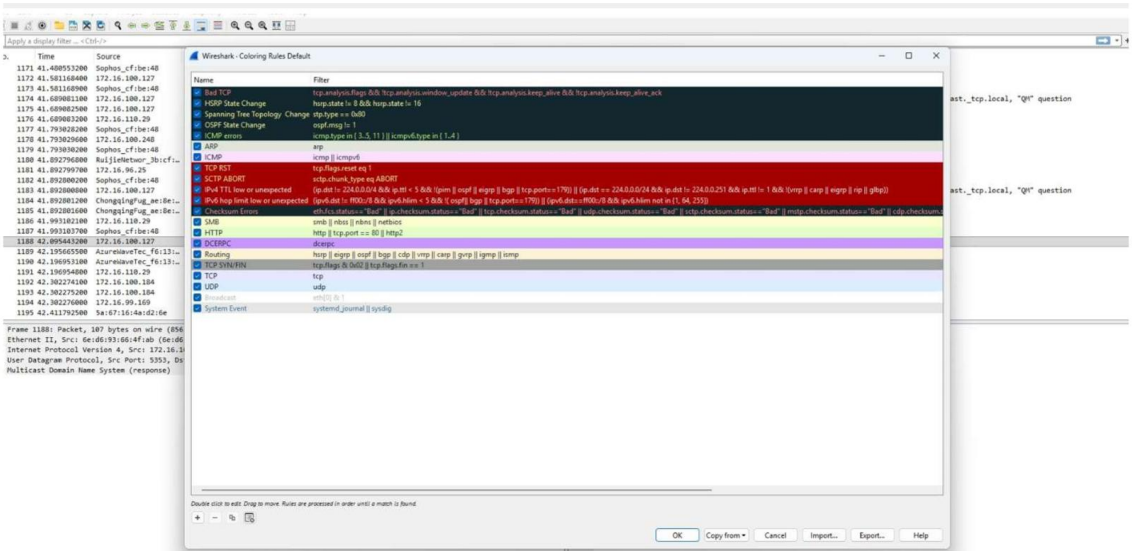
No.	Time	Source	Destination	Protocol	Length	Info
15	2.345	192.168.1.10	8.8.8.8	ICMP	74	Echo (ping) request
16	2.356	8.8.8.8	192.168.1.10	ICMP	74	Echo (ping) reply
17	3.347	192.168.1.10	8.8.8.8	ICMP	74	Echo (ping) request
18	3.359	8.8.8.8	192.168.1.10	ICMP	74	Echo (ping) reply
19	4.351	192.168.1.10	8.8.8.8	ICMP	74	Echo (ping) request
20	4.362	8.8.8.8	192.168.1.10	ICMP	74	Echo (ping) repl

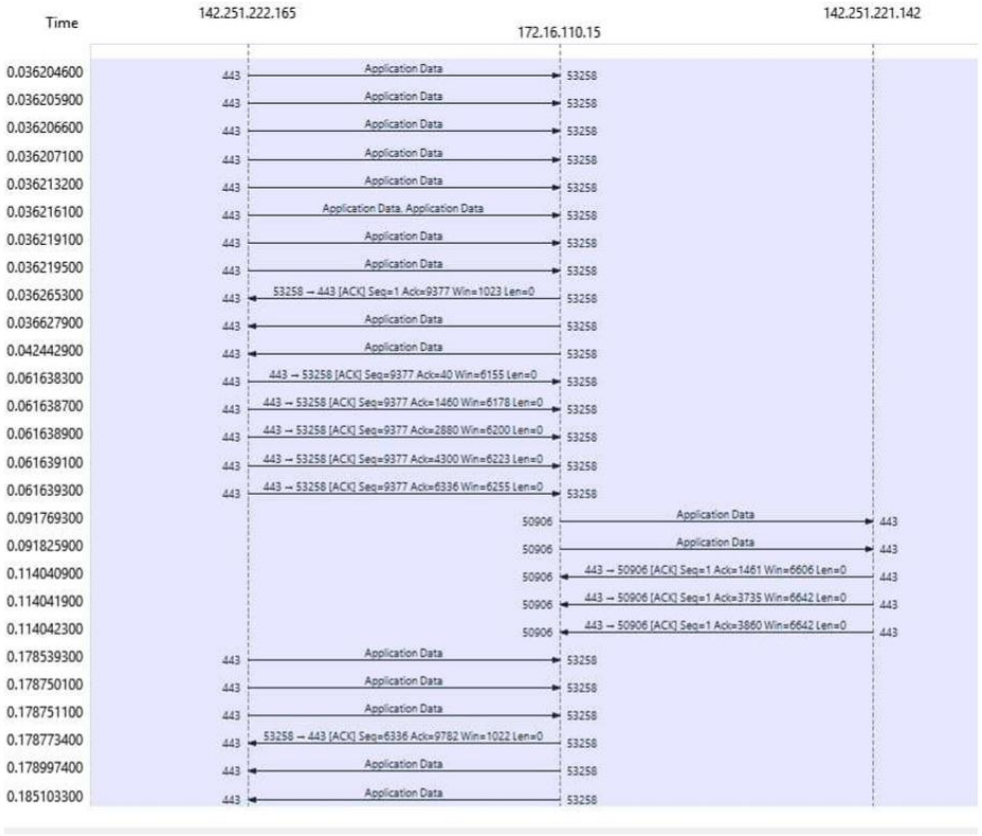
OUTPUT:






Wireshark · Coloring Rules Default
Name
Filter
Bad TCP
tcp.analysis.flags && !tcp.analysis.window_update && !tcp.analysis.keep_alive && !tcp.analysis.keep_alive_ack
HSRP State Change
hsrp.state != 8 && hsrp.state != 16
Spanning Tree Topology Change
stp.type == 0x80
OSPF State Change
ospf.msg != 1
ICMP errors
icmp.type in { 3..5, 11 } || icmpv6.type in { 1..4 }
ARP
arp
ICMP
icmp || icmpv6
TCP RST
tcp.flags.reset eq 1
SCTP ABORT
sctp.chunk_type eq ABORT
IPv4 TTL low or unexpected
(ip.dst != 224.0.0.0/4 && ip.ttl < 5 && !(pim || ospf || eigrp || bgp || tcp.port==179)) || (ip.dst == 224.0.0.0/24 && ip.dst != 224.0.0.251 && ip.ttl != 1 && !(vrrp || carp || eigrp || rip || glbp))
IPv6 hop limit low or unexpected
(ipv6.dst != ff00::/8 && ipv6.hlim < 5 && !(ospf|| bgp || tcp.port==179)) || (ipv6.dst==ff00::/8 && ipv6.hlim not in {1, 64, 255})
Checksum Errors
eth.fcs.status=="Bad" || ip.checksum.status=="Bad" || tcp.checksum.status=="Bad" || udp.checksum.status=="Bad" || sctp.checksum.status=="Bad" || mstp.checksum.status=="Bad" || cdp.checksum
SMB
smb || nbss || nbns || netbios
HTTP
http || tcp.port == 80 || http2
DCERPC
dcerpc
Routing
hsrp || eigrp || ospf || bgp || cdp || vrrp || carp || gvrp || igmp || ismp
TCP SYN/FIN
tcp.flags & 0x02 || tcp.flags.fin == 1
TCP
tcp
UDP
udp
Broadcast
eth[0] & 1
System Event
systemd_journal || sysdig
Double click to edit. Drag to move. Rules are processed in order until a match is found.
OK
Copy from
Cancel
Import...
Export...
Help


Wireshark · Coloring Rules Default
Name
Filter
Bad TCP
tcp.analysis.flags && !tcp.analysis.window_update && !tcp.analysis.keep_alive && !tcp.analysis.keep_alive_ack
HSRP State Change
hsrp.state != 8 && hsrp.state != 16
Spanning Tree Topology Change
stp.type == 0x80
OSPF State Change
ospf.msg != 1
ICMP errors
icmp.type in { 3..5, 11 } || icmpv6.type in { 1..4 }
ARP
arp
ICMP
icmp || icmpv6
TCP RST
tcp.flags.reset eq 1
SCTP ABORT
sctp.chunk_type eq ABORT
IPv4 TTL low or unexpected
IPv6 hop limit low or unexpected
Checksum Errors
SMB
HTTP
DCERPC
Routing
TCP SYN/FIN
TCP
UDP
Broadcast
System Event
Double click to edit. Drag to move. Rules are processed in order until a match is found.
OK
Copy from
Cancel
Import...
Export...
Help



RESULT:

Thus the experiments on packet capture tool in Wireshark is successfully observed and checked.